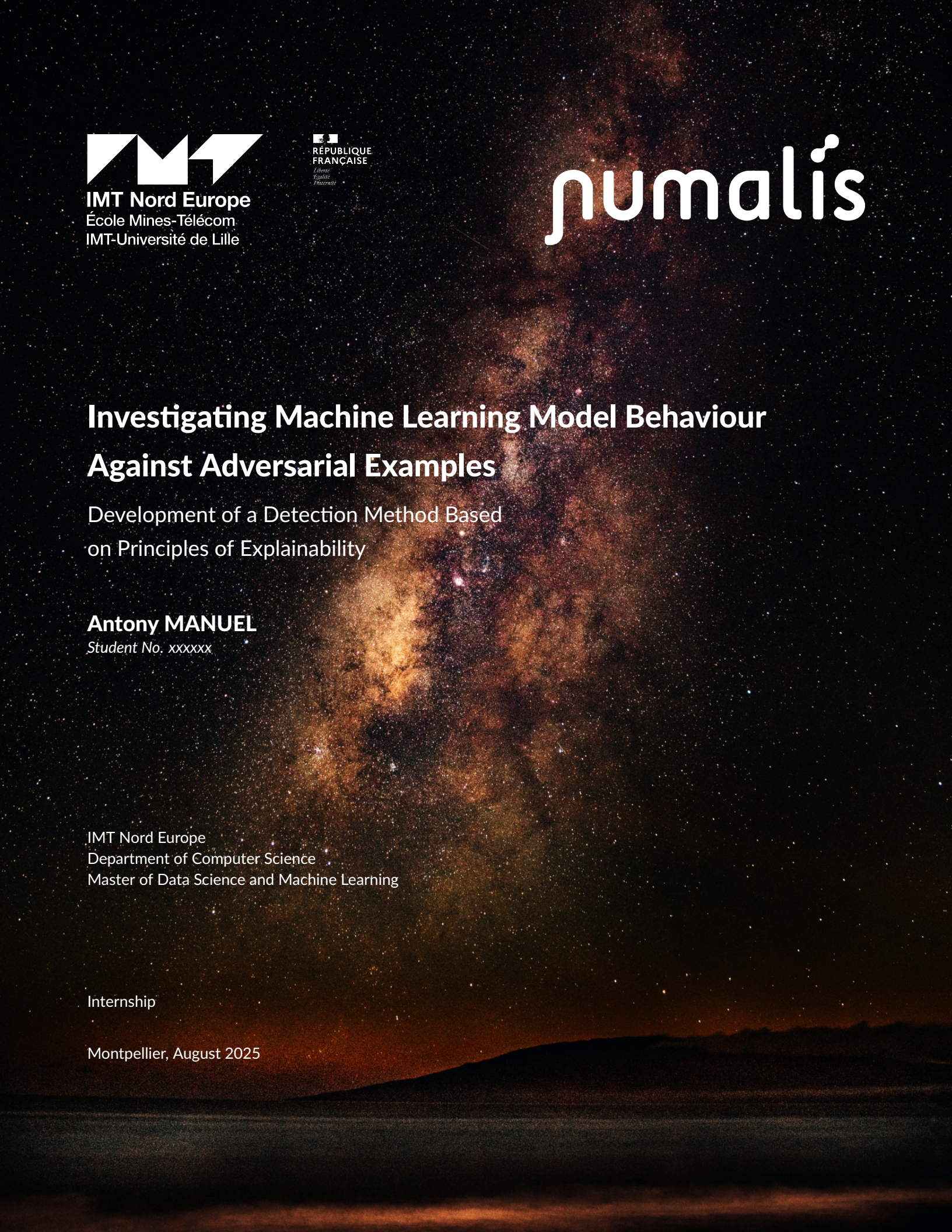


IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

RÉPUBLIQUE FRANÇAISE
Liberté Égalité Fraternité

numalis

Investigating Machine Learning Model Behaviour Against Adversarial Examples

Development of a Detection Method Based on Principles of Explainability

Antony MANUEL
Student No. xxxxxx

IMT Nord Europe
Department of Computer Science
Master of Data Science and Machine Learning

Internship

Montpellier, August 2025

IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

numalis

Investigating Machine Learning Model Behaviour Against Adversarial Examples

Development of a Detection Method Based on Principles of Explainability

Antony MANUEL
Student No. xxxxxx

Supervisor: Sofiane BESSAI
CIFRE PhD Student, Numalis

Co-supervisor: Jean-Pierre KEMPA
Associate Professor, IMT Nord Europe

Matthieu MARTEL
PhD Supervisor, Numalis, University of Perpignan

IMT Nord Europe
Department of Computer Science
Master of Data Science and Machine Learning

Internship

Montpellier, August 2025

Résumé

Ce rapport présente les travaux réalisés dans le cadre de mon stage de première année de master au sein de Numalis. Depuis plusieurs années, le Deep Learning a permis de réaliser des avancées majeures dans le domaine de lapprentissage automatique. Cependant, les exemples adversaires restent un défi majeur pour les modèles de machine learning, et ce, dans de nombreux domaines d'application. Ce travail vise donc à produire une méthode de détection des éléments adversaires, en proposant une approche basée sur des outils d'explicabilité, notamment l'outil CRAFT, développé par Fel et al. [1] qui permet d'observer ce que le modèle a perçu comme pertinent dans la classification d'un élément. Les résultats obtenus montrent une certaine efficacité, mais dépendent fortement de l'attaque utilisée. Cela ouvre ainsi des perspectives pour de nouvelles méthodes de défenses.

Mots clés : Réseaux de neurones, Exemples adversaires, Concepts, Factorisation de matrices non-négatives, Moindres carrés non négatifs

Abstract

This report describes the work carried out during the internship of the first year of my master degree at Numalis. For several years, Deep Learning has enabled major breakthrough in machine learning. Nevertheless, adversarial examples remain a major issue for machine learning models in many applications. This work aims to produce an adversarial examples detection method based on explainability. It relies on CRAFT framework developed by Fel et al.[1] which offer the opportunity to observe the elements a model qualifies as relevant in its prediction. Finally the results obtained illustrate balanced effectiveness, depending on the attack used to generate adversarial inputs. However, it paves the way to new defenses method relying on explainability.

Keywords: Neural Networks, Adversarial Examples, Concepts, Non-Negative Matrix Factorization, Non-Negative Least Squares

Remerciements

Je souhaite tout d'abord exprimer mes plus sincères remerciements à Sofiane et Matthieu pour m'avoir offert l'opportunité de réaliser ce stage de recherche, ainsi que pour l'accompagnement constant, l'expertise et le soutien dont ils ont fait preuve tout au long de cette expérience. La confiance et l'autonomie qu'ils m'ont accordées ont été déterminantes, me permettant de progresser tant sur le plan intellectuel que professionnel. Leurs connaissances approfondies, leurs conseils éclairés et leurs retours constructifs ont considérablement contribué à orienter mes travaux et à en renforcer la rigueur scientifique.

Je tiens également à remercier Paul, Perrine et Alice, membres de l'équipe R&D, pour les échanges enrichissants au cours des discussions informelles, des parties d'échecs et des déjeuners partagés. Ces moments conviviaux ont été particulièrement précieux, notamment dans les périodes de doute ou de fatigue, en favorisant ma motivation et mon équilibre tout au long du stage.

Je souhaite également exprimer ma gratitude à l'ensemble des équipes de Numalis pour l'accueil chaleureux qui m'a été réservé et pour leur disponibilité. Leur volonté de partager leurs connaissances, de prodiguer des conseils avisés et d'émettre des critiques constructives a permis de créer un environnement de travail stimulant et enrichissant, propice à mon apprentissage et au développement du projet.

Je ne pourrais évoquer ce stage sans exprimer ma profonde reconnaissance envers ma soeur Candyce, mes frères Natan et Glory, ainsi que ma mère, Loetitia, dont le soutien constant et les encouragements ont été précieux tout au long de ce stage, notamment lors des moments d'insomnie et de fatigue. Je souhaite également remercier Rahil, celle qui m'a accompagné tout au long de ce stage, qui m'a soutenu et aidé à garder l'équilibre durant les périodes difficiles et de doutes et dont je remercie la présence, la compréhension et les encouragements. Leur présence et leur appui ont constitué un véritable moteur pour mener à bien ce projet.

Je remercie enfin les relecteurs et correcteurs de ce rapport, dont les observations ont contribué à améliorer la clarté, la précision et la qualité de ce document.

Cette expérience au sein d'un environnement de recherche et d'innovation m'a permis non seulement d'acquérir de nouvelles compétences techniques et méthodologiques, mais également d'approfondir ma compréhension des enjeux scientifiques et technologiques propres à ce domaine. Elle constitue, sans aucun doute, une étape déterminante dans mon parcours académique et professionnel.

Notation

$\mathcal{X}$	Set.
$\mathbb{N}$	Set of integers.
$\mathbb{R}$	Set of real numbers.
$\subset$	Strict inclusion.
$\subseteq$	Inclusion with possible equality.
$\lvert \cdot \rvert$	Absolute value of a real number.
$\lVert \cdot \rVert$	Euclidean norm of a vector.
x	A real scalar belonging to $\mathbb{R}$.
$x \in \mathbb{R}^n$	A vector in $\mathbb{R}^n$.
$X \in \mathbb{R}^{n \times d}$	A matrix in $\mathbb{R}^{n \times d}$.
I_n	Identity matrix of size n.
$X^\top$	Transpose of X.
X^{-1}	Inverse of X.
x	A random scalar variable.
$x \in \mathbb{R}^n$	A random vector in $\mathbb{R}^n$.
$X \in \mathbb{R}^{n \times d}$	A random matrix in $\mathbb{R}^{n \times d}$.
$P(x)$, $P(\mathbf{x})$	Probability distribution of x (resp. joint probability distribution of $\mathbf{x}$).
$x \sim P_x$	Random variable x follows a probability distribution P_x.
$\mathbb{E}(x)$	Expectation of x with respect to $P(x)$.
$\mathrm{Var}(x)$	Variance of x under $P(x)$.
x_i	i-th element of the vector x.
$X_{i,j}$	Element at row i, column j of the matrix X.
$X_{i,:}$	i-th row of the matrix X.
$X_{:,i}$	i-th column of the matrix X.
$f \circ g$	Composition of functions f and g.
dy/dx	Derivative of y with respect to x.
$\partial y/\partial x$	Partial derivative of y with respect to x.
$\partial f/\partial x$	Jacobian matrix $J \in \mathbb{R}^{n \times p}$ of $f : \mathbb{R}^n \to \mathbb{R}^p$.

Sometimes we use a function f whose argument is a scalar but apply it to a vector $f(x)$ or a matrix $f(X)$. This denotes the element-wise application of f to the array.

$$* \ * \ *$$

Contents

Contents **5**

1 Introduction **7**
1 Company presentation 8
2 Machine Learning Background 9
2.1 Models 9
2.2 Neural Network 9
2.3 Adversarial example 10

2 First Approaches **12**
1 Project Overview 12
2 Preliminary Implementation 13

3 Technical Implementation **15**
1 CRAFT Background 15
1.1 Concepts 15
1.2 Non-negative Matrix Factorization 16
1.3 Sensitivity Analysis 17
2 Image Retrieval Pipeline 18
3 Concept analysis 19
3.1 Loss Derivative 21
3.2 Projection & Importance Estimation 23

4 Final Production **26**
1 Protocol Overview 26
1.1 General Architecture 26
1.2 Component Breakdown 26
1.3 Results & Evaluation 28
2 Areas for Improvement 29

5 Conclusion **30**

Bibliography **33**

A Appendix **35**

B Annexe **40**

Introduction

For nearly a century, technological progress has enabled the development of increasingly sophisticated machines with ever-expanding capabilities. In the present era, we are witnessing the pervasive integration of Artificial Intelligence (AI) into everyday life, accompanied by rapid advancements in its underlying methods. With each passing year, more powerful AI systems are introduced for mainstream adoption, consistently surpassing the abilities of their predecessors. These improvements are largely driven by enhanced computational resources, which allow Machine Learning (ML) algorithms to be deployed across a broad spectrum of commercial and professional applications. Public interest in these technologies has grown substantially, reflecting both fascination and optimism regarding their potential.

There is hardly a doubt that ML and particularly Deep Learning (DL) algorithms achieve remarkable performance across a variety of tasks, including object recognition, speech processing, and autonomous driving. Nevertheless, despite these achievements, our understanding of their true capabilities and internal mechanisms remains limited.This gap in knowledge has given rise to the research area of explainability. One critical challenge in this context concerns the ability of models to maintain performance when faced with inputs drawn from distributions that differ, even slightly, from the training data. This property is commonly referred to as generalisation beyond the training dataset, and, in the optimisation literature, as robustness. In this setting, a robust solution is one that continues to perform well under a certain degree of uncertainty [2].

Research in DL has revealed that Neural Network (NN) often display limited robustness, which has motivated a substantial and sometimes contentious body of work [3]. In particular, they can be highly sensitive to small, deliberate perturbations applied on inputs that remain visually or statistically similar to the training data yet cause severe performance degradation. Such inputs, first introduced in [3], are known as adversarial examples. Because adversarial examples are generated intentionally, they raise serious concerns regarding the security and reliability of deploying DL algorithms in critical applications. While debate continues about their practical implications, their very existence underscores the urgent need for a deeper understanding of NN behaviour in the presence of adversarial inputs and their overall robustness.

The following chapter intends to provide a clear introduction about the context of this research internship and a relevant background in machine learning, more specifically in

deep learning, in order to make the comprehension of this paper easier. The goal is not to make the state of the art intelligible, but only to give a few significant clue. The chapter is ordered as follows: section 1 produce an overview of the company, and on the other hand section 2 revisits the machine learning essentials.

1 Company presentation

Numalis is a small and medium-sized enterprise (SME) specializing in software validation and the development of design support tools. Established in 2015 by researchers Arnault Ioualalen and Matthieu Martel, the company was founded to industrialize the research initially conducted during Arnault's doctoral studies under Matthieu's supervision, which tackled floating point issue in numeric analysis. Building on this foundation, Numalis progressively developed the first formal validation tools for neural networks.

Historical industrial incidents emphasize the urgency of Numalis' mission. On February 25, 1991, an American Patriot missile deviated 600 meters off course due to a minor internal clock counter error, resulting in the death of 28 American soldiers in Dhahran, Saudi Arabia. Similarly, on June 4, 1996, the Ariane 5 flight 501 exploded 39 seconds after lift-off because the conversion of a 64-bit floating-point number to a 16-bit integer exceeded the latter's capacity. Such accidents highlighted the critical need for rigorous validation of computational systems and spurred research into methods to prevent and correct such failures.

Over two decades of dedicated research, Numalis has developed expertise in static code analysis via abstract interpretation, code synthesis, and abstract analysis, enabling the creation of highly reliable verification tools. Since its inception, in 2015, the company has focused on enhancing the reliability of critical computational code and providing confidence in software systems.

In 2017, Numalis expanded its activities to the validation of artificial intelligence, with support from the French Ministry of Defense (DGA). The company now addresses the challenge of ensuring the robustness and explainability of neural networks, particularly in safety-critical applications. The widespread adoption of deep learning in sectors such as aeronautics, automotive, healthcare, and defense is constrained by the lack of mature validation frameworks. As formal methods represent one of the most rigorous approaches to software verification, applying them to AI is a natural extension of Numalis' expertise.

The growing demand for reliable artificial intelligence underscores the critical importance of standardization within the field. In response, Numalis actively collaborates with the international AI community to advance the development of industry standards. Through its engagement with ISO/IEC and CEN CENELEC committees, the company contributes to the formulation of rigorous processes for AI validation, thereby supporting highly regulated industries in navigating complex regulatory and certification requirements.

While the underlying technologies continue to evolve, Numalis' foundational mission to deliver confidence, accuracy, and safety to its clients remains unwavering.

2 Machine Learning Background

2.1 Models

In the field of machine learning, a *model* can be defined as a computational framework, typically formalized through mathematical functions or algorithms, that aims to capture patterns and structures from data. During the *training* phase, the model adjusts its parameters iteratively in order to minimize an objective function, thereby learning a representation of the underlying data distribution. The ultimate goal is to generalize this acquired knowledge so that the model can successfully perform a specific task on previously unseen *test* data [2].

The quality of the learning process is commonly assessed through quantitative evaluation. Depending on the nature of the task, a variety of metrics can be employed, such as accuracy, precision, recall or F1-score. These metrics provide complementary perspectives on model performance and help to identify potential shortcomings.

Machine learning models span a broad spectrum of complexity. At one end of the spectrum, simple approaches such as linear regression or logistic regression rely on linear decision boundaries. At the other end, highly non-linear and ensemble-based methods—including decision trees, and deep neural networks—are capable of modeling complex, high-dimensional relationships within data.

Broadly defined, a machine learning algorithm learns a mapping function :

$$f : \mathcal{X} \to \mathcal{Y},$$

where $\mathcal{X}$ denotes the input space and $\mathcal{Y}$ output space.

In this paper, we focus specifically on *classification algorithms*, with particular emphasis on the task of *image classification*. Therefore, $\mathcal{X}$ is denoting images or extracted features and $\mathcal{Y}$ represents a discrete output space consisting of predefined labels. The objective of the classifier is to assign each input instance $x \in \mathcal{X}$ to its corresponding class $y \in \mathcal{Y}$ as accurately as possible [2].

2.2 Neural Network

A *neural network* is a machine learning model inspired by the structure and functioning of the human brain. It is composed of interconnected units, called neurons, organized into layers. Using the previously introduced notation, a neural network can be expressed as follows:

$$f = \sigma_k \circ \sigma_{k-1} \circ \cdots \circ \sigma_1$$

- σ_1 denotes the *input layer*, which receives the raw data.
- σ_i represents the *hidden layers*, which iteratively transform the data through successive computations.
- σ_k corresponds to the *output layer*, which produces the final prediction or classification.

Each layer σ_i applies both linear and non-linear transformations to its input. The non-linear component, known as the activation function, enables the network to capture complex patterns. Formally, each layer can be written as:

$$\sigma_i(\nu_{i-1}) = \phi(W_i^T \cdot \nu_{i-1} + b_i) = \nu_i \ ,$$

where ν_i and ν_{i-1} denote the outputs of the i-th and (i-1)-th layers, respectively, while W_i and b_i are the weight matrix and bias vector of the i-th layer. Common choices for the activation function ϕ include ReLU, tanh, and the sigmoid function. Neural networks learn by iteratively adjusting the weights and biases of each layer during training, typically via backpropagation, which minimizes the discrepancy between predicted and true outputs. The use of activation functions makes neural networks particularly powerful for modeling complex, non-linear relationships in data, thereby enabling their widespread application in tasks such as image recognition, natural language processing, and time-series forecasting.

2.3 Adversarial example

We are now ready to provide a formal definition of adversarial examples, originally introduced in [3]. Let f denote a neural network (NN) image classifier and let $x \in \mathcal{X}$ be a correctly classified input. An adversarial example is obtained by adding a minimal perturbation η to x, such that the perturbed input $x' = x + \eta$ is misclassified. Formally [2],

$$\begin{aligned} \min_{x'} \ \|x' - x\|_p = \min_{\eta} \ \|\eta\|_p \\ \text{s.t.} \quad f(x) = l, \\ f(x') = l', \\ l \neq l', \\ x' \in [0,1]^m, \end{aligned} \tag{1.1}$$

where $\|\cdot\|_p$ denotes the L_p distance, defined as

$$\|z\|_p = \left(\sum_i |z_i|^p \right)^{\frac{1}{p}} .$$

Intuitively, the optimization aims to identify the smallest possible perturbation, under a chosen norm, that alters the model's prediction. The p-norm is employed to quantify

imperceptibility: L_0 denotes the number of pixels that are modified, L_2 measures the standard Euclidean distance, and L_∞ captures the maximum change applied to any pixel, without constraining the number of pixels that may be altered.

There is a wide range of generation methods, as described in the paper by Serban et al. [2], as well as various defense methods. However, as shown in Shafahi et al. paper [4], defenses are often broken by more sophisticated attacks. In this work, we focus on the Fast Gradient Method (FGM), the DeepFool (DF), and the Carlini&Wagner (CW) [5].

First Approaches

1 Project Overview

During the first few weeks, I focused on building a solid understanding of the subject by reviewing several research papers. I paid particular attention to the strict definitions of Adversarial Examples, the various techniques used to generate them, and the current state-of-the-art defenses designed to mitigate their impact. Among the most notable attack methods I studied the CW attack, which is known for its ability to bypass many existing defenses through carefully optimized perturbations; the FGM, a widely used and computationally efficient technique that generates adversarial examples by applying a single step in the direction of the gradient; and DF, which iteratively perturbs inputs to move them across decision boundaries with minimal distortion. On the defense side, I explored techniques such as adversarial training, which enhances model Robustness by including adversarial examples in the training process; defensive distillation, which reduces model sensitivity to small input changes; and various forms of input preprocessing designed to detect or neutralize adversarial noise. This foundational research allowed me to better understand both the vulnerabilities of machine learning models and the strategies used to reinforce their security.

Initially, the method proposed by Sofiane involves selecting relevant layers of the neural network and analyzing their outputs to construct reference tensors. For a given input x, its inference profile is extracted and compared to the reference profiles using a selected metric. Based on a predefined threshold, the input is then classified as adversarial or benign.

The primary objective of the internship was to investigate the limitations of the method developed by Sofiane [6]. More specifically, my task was to explore the influence of various hyperparameters that could significantly affect the method's performance. These include the decision threshold, the number of reference elements used for comparison, as well as other key parameters.

2 Preliminary Implementation

After developing a strong foundation in the subject, I was given the opportunity by my supervisors to explore and experimentally investigate the ideas I had been formulating. I used CIFAR10 and CIFAR100 dataset to do my experimentation, and defined two NNs, for my experimentation, for each of these dataset. My first strategy was to seek anomalies between an adversarial example and elements from its original class and later with its predicted class. So in the first place, I defined the similarity based on the cosine similarity, defined as follows :

$$\forall x, y \in \mathbb{R}^n, \quad \text{cosim}(x, y) = \frac{\langle x, y \rangle}{\|x\|_2 \cdot \|y\|_2} \tag{2.1}$$

This value, is between -1 and 1, the more the value is close to 1, the more similar are the two vectors given.

I computed the similarity between each layers output of each element of the same class and plotted the average cosine simiilarity order by layers in the following figure : These figures illustrate the fact that, in the middle of the network it was really difficult to

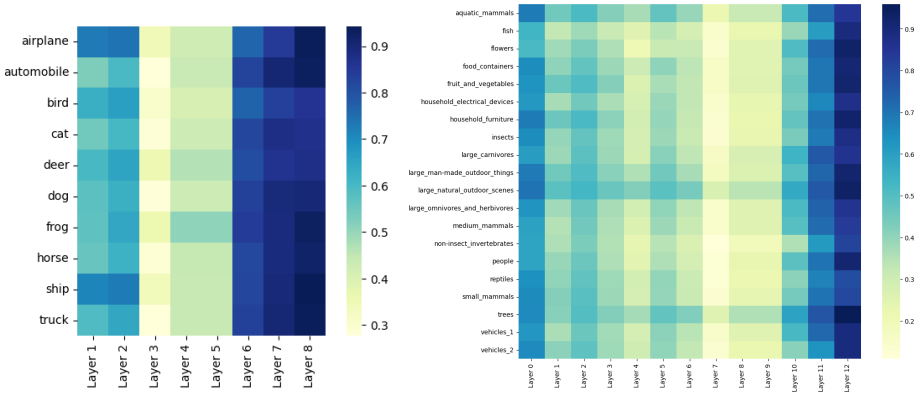


Fig. 2.1: **Cosine similarity between element from the same class.**

retrieve information specifically the similarity between elements, while more information were contained at the end of the network, specifically at the penultimate layer. As a result, I decided to only focus on the three last layers before the output. However, the output of those layers are tensors of high dimension, for this reason I decided to use a PCA to address the issue and to be more focus on important information.

Moreover, after this first experimentation, I decided to select three layers, the last three before the output, and defined an average activation tensor by class and compared each tensor of generated adversarial example with it. The coming figure, illustrates, the similarity between the activation tensor of each class and the tensor of an adversarial element, for three layers.

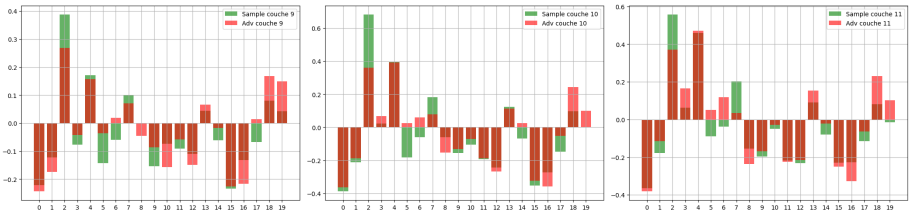
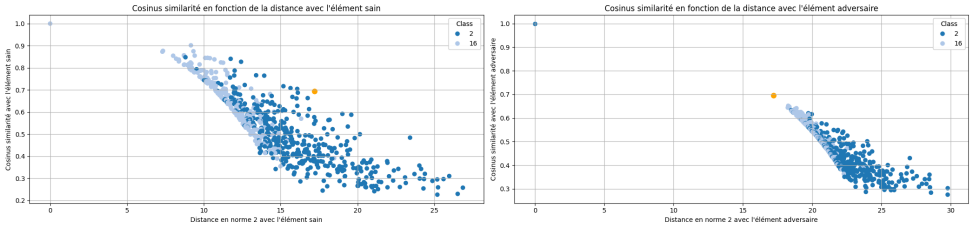


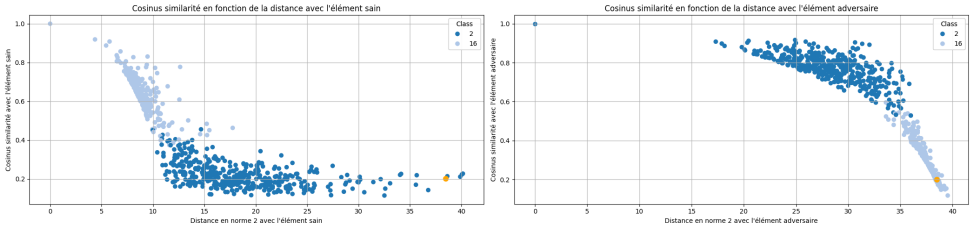
Fig. 2.2: **Cosine similarity with average tensors.**

I observed for each class, there was not a distinct behavior, between the sain element' tensor and the adversarial element generated, regarding to similarity through the different layers.

At the moment I was not really inspired about interpretation that could possibly made from this, subsequently I decided to examine more precisely the conduct of an adversarial example with regard to all elements of a class, in particular the initial class and the one predicted. This process led me to plot the evolution of the the distance and the similarity of a sample and an adversarial example generated from it through all the layers. The successive illustration can be found in the appendix A.1, but we will only focus on the first and the penultimate layers. The Fig 2.3 is highlighting the evolution of the distance by



(a) First layer output comparison



(b) Penultimate layer output comparison

Fig. 2.3: **Cosine by distance**

the cosine similarity of a benign sample and adversarial example. It can be said that due to the link between the cosine similarity and the distance there is

Technical Implementation

1 CRAFT Background

At the close of those first analyses, I went around explainability and Thomas Fel's work. The following section aims to present the main points that arouse my interest and constitute the basis of my work.

We need to consider now, a supervised learned predictor $f : \mathcal{X} \to \mathcal{Y}$ that can be separated into two different function, and can be expressed for an input $\mathbf{x} \in \mathcal{X}$ as $f(\mathbf{x}) = (h \circ g)(\mathbf{x})$. The first one, g, is mapping the input $\mathbf{x}$ to an intermediate state $g(x) \subseteq \mathbb{R}^p$ in a latent space, and the second one, h, is combining this intermediate state to the output.

1.1 Concepts

The rapid advancement of deep learning models is matched by the growing range of their applications. As previously highlighted, this progress makes it increasingly important to establish trust in these models, trust that must be grounded in both understanding and control. This need has given rise to the field of Explainable Artificial Intelligence (XAI), which aims to develop methods that provide insight into model behaviour. Among the various branches of XAI, concept-based explainability focuses on understanding what the model perceives, rather than merely identifying where it is focusing. This distinction underpins the development of concept-based XAI methods, which seek to interpret internal model representation in terms of high-level, human-understandable concepts. One such method is CRAFT, introduced by Fel et al. [1], which exemplifies this approach. However, prior to the work of Poeta et al. [7], the notion of a concept in this context lacked a consistent definition, resulting in a somewhat ambiguous and fragmented understanding across the literature. To address this, Poeta et al. conducted a comprehensive survey of concept-based explainability methods and provided a clearer taxonomy for the term *concept*. They proposed four main categories : *Symbolic Concepts, Unsupervised Concept Bases, Prototypes* and *Textual Concepts*.

In the context of the CRAFT method, the relevant definition falls under the category of *Unsupervised Concept Bases*. According to Poeta et al., these refer to "clusters of data samples learned by a neural network without relying on human-provided labels. Such clus-

ters emerge within the model's latent space and capture abstract patterns or features that, while not explicitly defined by humans, may correspond to human-interpretable concepts".

1.2 Non-negative Matrix Factorization

Now that we have established a clear definition of a "concept", we proceed to examine its materialization within our proposed method.

Let us consider a dataset of n images $(x_1, \ldots, x_n) \in \mathcal{X}^n$ and their corresponding labels $(y_1, \ldots, y_n) \in \mathcal{Y}^n$. In order to explain a specific class $y \in \mathcal{Y}$, we extract the subset of images from the dataset that are associated with the label y by the model. This subset is denoted as:

$$\mathcal{S}_y = \{x_i \mid f(x_i) = y, \forall i \in [\![1; n]\!]\}.$$

A transformation function $\pi(\cdot)$ is then applied to each image in $\mathcal{S}_y$, producing a new set of localized image regions:

$$\mathbf{X}_y = \{\pi(x_i) \mid x_i \in \mathcal{S}_y\}.$$

To identify distinct concepts within these localized regions, we compute their activations via the function g, resulting in the activation matrix $\mathbf{A} = g(\mathbf{X}_y) \in \mathbb{R}^{n \times k}$. Subsequently, Non-negative Matrix Factorisation (NMF) is applied to identify a basis of interpretable concepts by solving the following optimization problem:

$$(U, B_y) = \underset{U \geq 0,\, H \geq 0}{\arg\min} \frac{1}{2} \|\mathbf{A} - U B_y^\top\|_F^2, \tag{3.1}$$

where $U \in \mathbb{R}^{n \times r}$, $B_y \in \mathbb{R}^{k \times r}$, with $r \ll \min(n, k)$, and $\| \cdot \|_F$ denotes the Frobenius norm.

NMF is a widely used technique in various domains such as natural language processing (e.g., text classification), image processing, and feature extraction. Compared to alternative factorization methods such as Single Value Decomposition (SVD) or Principal Component Analysis (PCA), NMF offers greater interpretability due to its non-negativity constraint. Specifically, NMF enables the interpretation of each input as a composition of latent, additive, and non-negative components, referred to as "concepts". The strict positivity of the decomposition terms further enhances interpretability. Each image crop can thus be viewed as a superposition of concepts. Indeed, solving the factorization results in two matrices: the *concept value matrix* U, and the *concept basis* or *concept bank* B_y. More precisely, the rows of U represent the coordinates of $\mathbf{A}$ in the new $\mathbb{R}^r$ basis defined by B_y, which maps from the original $\mathbb{R}^k$ space. Each localized crop $\pi(x_i)$ is therefore expressed as a linear combination of concepts:

$$\mathbf{A}_i = \sum_{j=1}^{r} U_{i,j} (B_y^\top)_j.$$

Due to the limited resolution of CIFAR-10 images (32 × 32 pixels), interpreting the localized crops in a human-understandable way remains challenging. Examples of such crops are illustrated in the following figure:



(a) Best crops of the most important concept of plane class



(b) Original images of the previous crops

Fig. 3.1: **Illustration of CRAFT concepts.**

While other decomposition techniques, such as SVD or Independant Component Analysis (ICA), have been employed for similar tasks, NMF remains both computationally efficient and effective at producing semantically meaningful components.

It is important to note, however, that the NMF decomposition is not unique. For any invertible matrix $M \in \mathbb{R}^{r\times r}$, the pair $(UM, M^{-1}B)$ yields an equivalent factorization. This non-uniqueness can lead to instability in the learned components, potentially affecting the interpretability and reproducibility of the results, especially in applications where consistent semantic meaning across runs is critical.

1.3 Sensitivity Analysis

After constructing the concept bank, a natural next step is to identify which concepts are relevant and exert a meaningful influence on the model's output. To address this, Fel et al. employed sensitivity analysis as a means of quantifying concept importance. In this subsection, we describe and define the "total Sobol indices" utilized in their approach and explain how these indices are applied to assess the impact of individual concepts and their interactions on the model's output variance.

According to Fel's doctoral thesis, a natural way to estimate the importance of a given concept i is to evaluate the fluctuations of the model's output $h(UB^{\top})$, in response to meaningful perturbations of the concept coefficient $U_{:,i}$. In practice, they define a perturbation masks $M = (M_1, \cdots, M_k) \sim \mathcal{U}([0,1]^r)$, which is an independant and identically

distribued (iid) vector of real-valued random variables. They introduced a concept fluctuation to generate a perturbed activation $\mathbf{A} = (U \odot M)B^\top$ where $\odot$ denote the Hadamard product (e.g., the masks can be used to remove a concept by setting its value to zero). Then this perturbed activation can be propagate to the model output in order to have the associated random output $Y = h(\mathbf{A})$. They explained, in simpler terms that perturbing or removing a cocnept that plays a significant role in the model will lead to a substantial change in the output, whereas modifying an unused or irrelevant concept will result in minimal impact.

To formally quantify the importance of a given concept, as isolated but also through its interactions with other concepts, they consider the expected variance in the model's output when all concept are fixed except the i-th, which leads to the general formulation of the *total Sobol indices* for concepts.

$$S_i^T = \frac{\mathbb{E}_{M_{\sim i}}\left(\mathrm{Var}_{M_i}(Y \mid M_{\sim i})\right)}{\mathrm{Var}(Y)} \tag{3.2}$$

2 Image Retrieval Pipeline

I conducted a series of experiments to investigate the potential relationships between the previously defined *concepts* and the classification outcome of an adversarial example. First, I extracted the *penultimate output* of every element of the training dataset and subsequently computed a PCA. For each class, I then selected the relevant concept, which was, as a first step, the most important one, and retrieved the original images associated to the most important crops that contributed to the generation of the latter concept. These images serve as reference examples for each class.. For example, Fig. 3.1 presents representative reference images corresponding to the *airplane* class. Following this, I extracted the *penultimate output* for each of these images and applied the previously computed PCA instance, storing the resulting outputs, which are considered as the most relevant tensor for each class, for further analysis. Those results are for each class set as the *reference tensors* and will be marked as follows $\mathcal{X}_{\mathrm{Ref},i}$.

Finally, the behavior of test elements was analyzed with respect to the reference tensors. For each test element $x_{\mathrm{test}} \in \mathcal{X}_{\mathrm{test}}$, the penultimate output representation $\nu_{\mathrm{test}}^{l-1}$ was extracted and projected using the principal component analysis (PCA) instance previously computed, resulting in the transformed representation ν_{PCA}.

The similarity and distance between ν_{PCA} and each class's reference tensor, $t_{\mathrm{Ref},k} \in \mathcal{X}_{\mathrm{Ref},i}$, were computed. Later on, the mean similarity between the test element and the reference tensors for each class was calculated. These mean similarity values were then sorted to facilitate comparison across classes. This process enabled the evaluation of whether the reference tensors associated with the predicted class or those of the true class exhibit a greater degree of similarity or proximity to the test element.

Tab. 3.1: **Top-3 Match Rate between test elements and reference tensors**

Metric	Predicted Class	True Class
Top-3 Match Rate (%)	0.8504	0.7987

The previous figure, Table 3.1, summarizes the statistical analysis of the similarity between test elements and the reference tensors corresponding to each class. In 0.85% of the cases, the predicted class was among the three most similar classes to the test element. Similarly, in 0.80% of the cases, the true class of a misclassified element was also among the top three most similar classes.

The descriptive statistics presented in Table 3.2 provide insight into the rank difference between the predicted class and the most similar class, as well as between the true class and the most similar class, based on similarity measures.

Tab. 3.2: **Descriptive statistics between test elements and reference tensors**

Metric	Predicted Class	True Class
Mean	0.9106	0.7768
Variance	1.5149	1.2196
Standard Deviation	1.2308	1.1044

This criterion was subsequently applied to adversarial examples. However, the approach proved insufficient to effectively distinguish adversarial samples from benign ones. Only approximately half of the generated adversarial examples were successfully detected.

3 Concept analysis

Thereafter, I sought another discriminative criteria, especially connected to the concepts extracted earlier. Initially, inspired by CRAFT implementation, my approach was to construct a bank of concept over the entire dataset. The objective was to identify concepts that are either irrelevant to any class or whose overall contribution to predictions is negligible. To build this concept bank, I concatenated the concept bank matrix $B_y \in \mathbb{R}^{5\times 64}$ from each class into a single matrix $H \in \mathbb{R}^{50\times 64}$.

Once, the bank of concept was build, I have used L2 distance and cosine similarity metrics to analyse those concepts, in order to note similar concept or totally different concept. The L2 distance and cosine similarity are defined as follows:

$$\forall i, j \in [\![1; 50]\!], \quad \mathrm{d}_2(H_i, H_j) = \|H_i - H_j\|_2 \tag{3.3}$$

$$\forall i, j \in [\![1; 50]\!], \quad \mathrm{cosim}(H_i, H_j) = \frac{\langle H_i, H_j \rangle}{\|H_i\|_2 \cdot \|H_j\|_2} \tag{3.4}$$

where $\forall k \in [\![1; 50]\!]$, H_k is the k-th rows of the bank concept matrix which represent a concept. As said above, it might indicate whether there is same or at least similar concepts, for the model, in the bank but it could emphazise otherwise because it is important to recall that the concepts extracted could be hardly interpretable by humans and still remain important in the classification process if they mean something for the model.

I then applied Sobol indices, as proposed in [1], to compute the importance of each concept with respect to the prediction of a given class. At this stage, I encountered a significant challenge during my internship. Indeed, I observed a concerning behavior: the concepts represented by the rows $H_{i,:}$ of the matrix H, particularly those appearing earlier in the matrix, in other words the former lines, consistently exhibited higher importance scores, regardless of their actual semantic relevance. Even after shuffling the rows of H, this positional bias persisted, indicating that indices was overly sensitive to the ordering of the concepts in the matrix, rather than their content. I attempted to analyze the implementation of Sobol's indices to understand the root cause of this issue, but was unsuccessful. I also explored alternative approaches; however, due to time constraints, the need to deliver results during my internship, and limitations in my mathematical background, I was unable to solve the problem satisfactorily.

Furthermore, up to this point, the majority of my implementations had been carried out using the TensorFlow framework, along with the integration of TensorFlow implementation of the CRAFT and ART libraries. However, the combined use of these two libraries proved to be suboptimal. In particular, the ART library required disabling TensorFlow's eager execution mode, a constraint that introduced several complications. Eager execution in TensorFlow allows operations to be evaluated immediately as they are called, enabling intuitive and interactive model development. It facilitates dynamic control flow, straightforward debugging, and seamless integration with Python and NumPy, making it particularly suitable for research and prototyping. Disabling eager execution reverts TensorFlow to its graph-based execution model, which, while more efficient in production environments, introduces several limitations. Chief among these is the incompatibility between tensors created in eager and non-eager modes, which prevents their interchangeability and can result in runtime errors or silent failures, further limiting the flexibility and consistency of the workflow. Additionally, managing variables and execution flow becomes more cumbersome in graph mode, increasing the risk of inconsistencies and making debugging more complex. These constraints significantly hinder experimentation and analysis, particularly when libraries or methods rely on functionalities exclusive to eager execution. As a result, the need to disable eager mode imposed by the ART library led to an inflexible development process and ultimately compromised the reliability and reproducibility of the experimental results. Notably, this requirement necessitated training the model twice, which adversely affected the outcomes derived from the CRAFT instance and impeded the analysis of adversarial behavior in relatin to concepts.

Consequently, due to the constraints imposed by disabling eager execution and the resulting incompatibilities, I made the decision to transition to PyTorch an alternative

framework better suited to the requirements of my experimental setup and future experimentation. With PyTorch framework, I have been able to adress those issues.

3.1 Loss Derivative

Before providing a detailed explanation and performing the associated calculations, let us first outline some key definitions. Let $x \in \mathbb{R}^{32\times32\times3}$ represent the input when using the CIFAR-10 dataset, and $y \in \mathbb{R}^{10}$ denote the output of the neural network, where 10 corresponds to the number of classes in the dataset. Furthermore, we define the output of the penultimate layer as $x^{(l-1)} \in \mathbb{R}^{64}$, where l represents the total number of layers in the network.

With a view to better understand the effect of different concepts on model predictions, I aimed to analyze the evolution of the loss function with respect to the bank of concepts. In practice, to examine loss function evolution, I had to define two neural networks with the exact same random weights, train the first one and extract concepts from it, build the bank of concepts and then analyses loss function with the second NN through the cross entropy derivative over the concepts matrix. The loss function used to train the NN was the cross entropy, defined as follows :

$$L(y, \hat{y}) = -\sum_{i}^{c} \hat{y}_i \cdot log(y_i), \tag{3.5}$$

We are now positioned to formally define the derivative of the loss function with respect to the concept bank. Upon solving the NNLS optimization problem presented in Equation 3.12, using $x^{(l-1)}$ as the input, the following can be stated:

$$x^{(l-1)} = Z \cdot H, \tag{3.6}$$

where $Z \in \mathbb{R}^{50}$ is the concept value matrix and $H \in \mathbb{R}^{50\times64}$ the bank of concept, however with a linear classifier, the output y can be expressed as follows:

$$y = x^{(l-1)} \cdot W, \tag{3.7}$$

where $W \in \mathbb{R}^{64\times10}$ represents the weight matrix at the penultimate layer. Therefore, we define the weighted bank of concepts matrix as $W' \in \mathbb{R}^{50\times10}$, which leads to an output that can be expressed as follows :

$$\begin{aligned} y &= (Z \cdot H) \cdot W \\ &= Z \cdot (H \cdot W) \\ &= Z \cdot W' \end{aligned} \tag{3.8}$$

where $y, \hat{y} \in \mathbb{R}^{10}$ are respectively the predicted probability vector and the ground-truth one-hot vector, and c the total number of classes.

Moreover, the previous definition of the loss is valid for a linear classifier, but if a non linear function such as softmax is applied, the cross entropy function can be written as follows :

$$L(s, \hat{y}) = -\sum_{i}^{c} \hat{y}_i \cdot log(s_i), \text{ with } s_i = \frac{e^{y_i}}{\sum_j^c e^{y_j}}$$

Finally, for both the linear and softmax classifiers, we define the partial derivative of the loss function with respect to the concept value matrix as follows :

$$\frac{\partial L}{\partial Z} = \begin{pmatrix} \hat{y}_1/y_1 \\ \vdots \\ \hat{y}_c/y_c \end{pmatrix} \cdot (W')^{\mathrm{T}} \tag{3.9}$$

$$\frac{\partial L}{\partial Z} = (s - \hat{y}) \cdot (W')^{\mathrm{T}} \tag{3.10}$$

The outcome of this equation 3.10 is an element of $\mathbb{R}^{50}$, where each value corresponds to a concept. These values were then plotted across each batch and epoch, resulting in the following visualization :

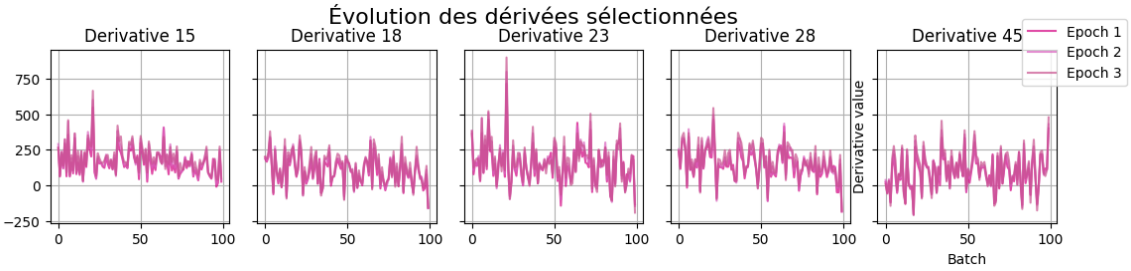


Fig. 3.2: **Loss derivative examples.**

For the sake of clarity, only a subset of the concept values is presented here, covering three epochs and one hundred batches. The full evolution of all values across a single epoch is provided in the appendix for completeness.

As illustrated in Fig. 3.2, while the overall temporal evolution of the derivative values appears relatively stable across epochs, notable differences in the amplitude of variation are observed among different derivatives. Specifically, while some derivatives remain below 800 across all epochs, others reach values as high as 4000, a disparity spanning up to three orders of magnitude.

Interpreting and understanding the underlying causes of these discrepancies proved challenging. Although I was unable to offer a definitive explanation, one plausible hypothesis is that derivatives exhibiting smaller variations may reflect more stable behavior across batches and could be more consistently aligned with different class representations.

Due to my limited background in the CRAFT method and the associated mathematical foundations, a full understanding of these dynamics was beyond my current expertise. Nonetheless, I believe the observed patterns are meaningful and may offer valuable insights into model explainability.

3.2 Projection & Importance Estimation

To better understand the effect of concepts on class predictions, and once the concept bank $H \in \mathbb{R}^{50\times 64}$ is computed, whit any new data sample x we transform the input with the function $\pi(\cdot)$ to obtain a set of patches $\mathcal{P}_x = \{\pi(x)\}$. In our case, with the CIFAR10 dataset, the current images size is (32, 32) then $Card(\mathcal{P}_x) = 9$. Afterwards, we compute its activations $\mathbf{A}_x$ via the function g as $\mathbf{A}_x = g(\pi(x))$. Finally we apply the underlying Non-negative Least Squares (NNLS) problem

$$\min_{u\geq 0} \|\mathbf{A}_x - uH\|_F^2 \ , \tag{3.11}$$

in order to determine the concept coefficients $u \in \mathbb{R}^{9\times 50}$ in the concept basis H.

Specifically, I aimed to analyse the decomposition of both a clean sample and an adversarial example derived from it, also look over the differences between elements from different classes. The following figure illustrates the value of the different lines of u associated to a benign input x (on the left) and an adversarial example associated x_{adv} (on the right). Each lines corresponding to a patch of x and x_{adv} are referred with distinct colors.

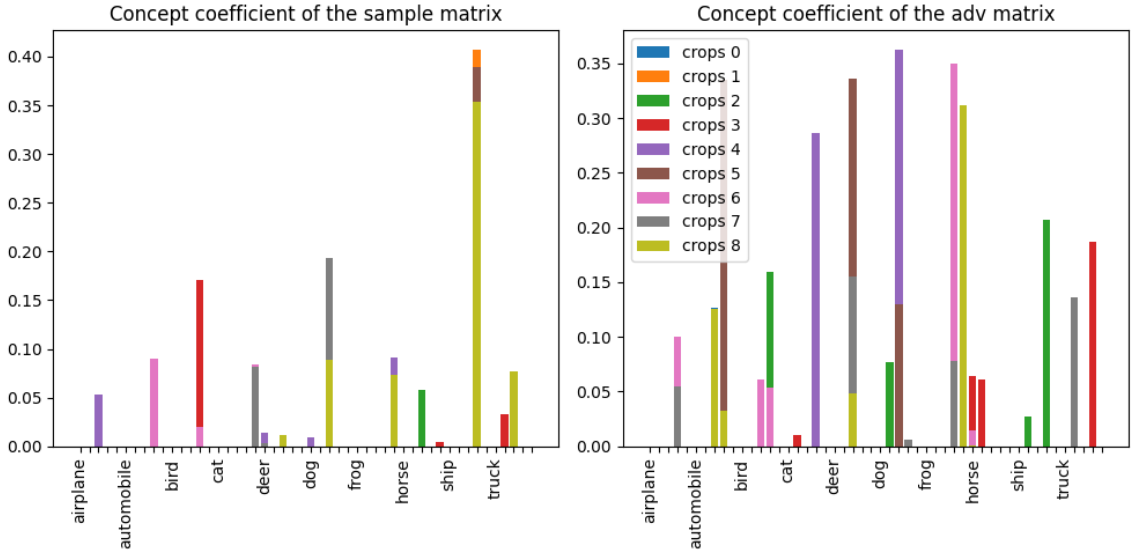


Fig. 3.3: **Projection comparison.**

Based on [1] the previous projection could be presented slightly differently as they proposed to solve the following NNLS problem for any new input x (which is a full image),

$$\min_{u \geq 0} \frac{1}{2} \|g(x) - uH^\top\|_F^2 \ , \tag{3.12}$$

and therefore recover its decomposition in the concept basis with concept coefficients u associated. Unfortunately, I did not have the opportunity to analyze the differences between the two definitions of the NNLS problem, which may warrant further investigation.

Furthermore, I applied the method used for estimating concept importance in the CRAFT instance to assess the relative importance of the concepts in the concept bank H with respect to each class in the dataset. This approach involves computing the total Sobol indices, as described in Section 1.3. The results of this analysis illustrates the relative importance of each concept in predicting elements classified under the class $y \in \mathcal{Y}$. The figure 3.4 is an example of this for the "airplane" category. A correlation with the loss

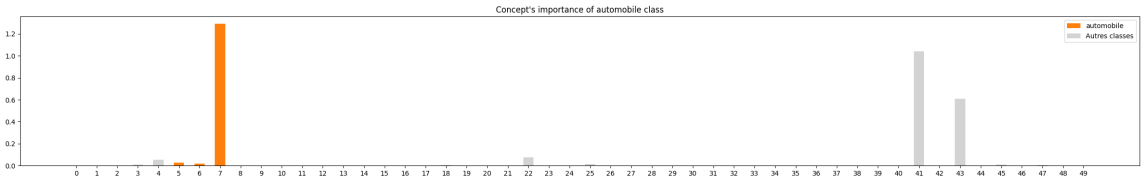


Fig. 3.4: **Importance of concepts for a class.**

derivative figure was expected, between the concept highly important and those causing a stable fluctuations or a notable variations of the loss function, but no discernible relationship emerged. This method could be extended by applying it to all elements that the model classifies as belonging to a given class y_k across the entire dataset. Specifically, this involves incorporating elements misclassified from other classes y_j, where $j \neq k$, and excluding elements from class y_k that are themselves misclassified, in order to maybe improve the effectiveness of this method.

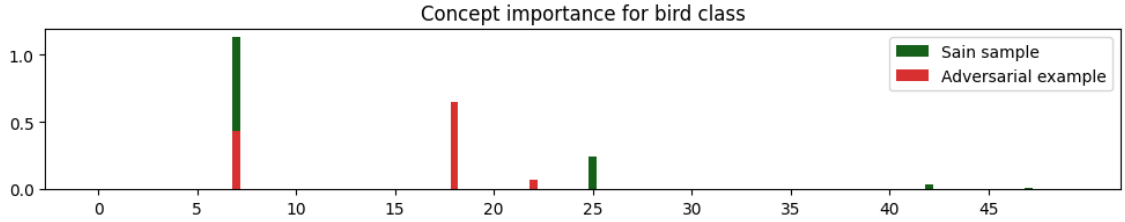


Fig. 3.5: **Example of concepts importance for a an input and an adversarial example.**

Additionally, I also tried to discover a difference between the result of the Sobol indices for a sain input and an adversarial one. Afterwards, I computed the importance estimation

method for the sain sample x_{sain} and the adversarial one x_{adv}. The follwing figure Fig .3.5 illustrate the comparison of importance estimation of concepts in order for the model to predicted those two input in the bird class. The results of the estimation for all classes are presented in the Fig. A.3 in the Section A.

While these results are insightful, they are not sufficient to establish a discriminative criterion on their own. Additional information is required to enhance their interpretability and relevance. The following section presents complementary results, which could be considered collectively for a more comprehensive analysis.

Final Production

1 Protocol Overview

This chapter introduces the adversarial detection method developed in this study, designed to address aversarial attacks, and provides a comprehensive analysis of the experimental results. The following sections will discuss the method's underlying approach and present its performance metrics.

1.1 General Architecture

The general architecture of the proposed protocol is outlined as follows. A balanced subset of elements is sampled from the training dataset used as sain elements and an other selection is used for adversarial generation. Then, we have a new set $\mathcal{S}$ of sain and adversarial elements generated with FGM, DeepFool or CW attack, such that

$$\mathcal{S} = \{\widehat{x}_i, \forall i \in \mathcal{I} \subset \mathbb{N}\}$$

Each selected input is projected onto the concept basis H, resulting in a corresponding set of coefficients $\{U_i\}_{i\in\mathcal{I}}$, where each U_i represents the decomposition of an input in the space defined by H. This transformation yields an auxiliary dataset $\mathcal{U} = \{(U_i, \widehat{y}_i)\}_{i\in\mathcal{I}}$, where $\widehat{y}_i$ denotes whether the elements is sain or an adversarial one.

Subsequently, two machine learning models, a Support Vector Machine (SVM) and a Random Forest, are independently trained on the auxiliary dataset $\mathcal{U}$. These models leverage the transformed representations to perform classification tasks.

1.2 Component Breakdown

Adversarial generation Adversarial examples are perturbed inputs designed to mislead machine learning models, while remaining nearly indistinguishable from the original data to human observers. This section discusses three widely used adversarial attack methods: the FGM, DeepFool, and the CW attack, that we use to test the robustness of this method.

The *Fast Gradient Method*, introduced by Goodfellow et al. [8], is a single-step attack that linearizes the loss function around the current input and perturbs the input in the

direction of the gradient to maximize the loss. Given a neural network model $f(\cdot;\theta)$, an input sample $x \in \mathbb{R}^n$, true label y, and loss function $J(\theta, x, y)$, the adversarial example x_{adv} is computed as:

$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}\left(\nabla_x J(\theta, x, y)\right), \tag{4.1}$$

where ϵ controls the perturbation magnitude. This method is computationally efficient but may produce larger perturbations compared to more advanced techniques.

DeepFool, proposed by Moosavi-Dezfooli et al. [9], is an untargeted, iterative attack that estimates the minimal perturbation required to change the classification result. It assumes the classifier is approximately linear in a small neighborhood around the input and computes the shortest path to the decision boundary. For a classifier $f : \mathbb{R}^n \to \mathbb{R}^K$, DeepFool updates the input iteratively using:

$$x_{i+1} = x_i + \frac{|f_k(x_i) - f_l(x_i)|}{\|\nabla f_k(x_i) - \nabla f_l(x_i)\|_2^2} \cdot \frac{\nabla f_k(x_i) - \nabla f_l(x_i)}{\|\nabla f_k(x_i) - \nabla f_l(x_i)\|_2}, \tag{4.2}$$

where f_k and f_l are the logits corresponding to the current and the next class boundary. The algorithm terminates once the classifier output changes. DeepFool generally produces smaller perturbations than FGM, making adversarial more complicate to detect.

The *Carlini & Wagner (C&W)* attack [5] formulates the adversarial example generation as a constrained optimization problem. It minimizes the perturbation required to change the model's prediction while ensuring that the adversarial input remains valid. The optimization problem is given by:

$$\min_{\delta} \|\delta\|_p + c \cdot L(x + \delta), \tag{4.3}$$

subject to $x + \delta \in [0, 1]^n$, where δ is the perturbation, $\|\cdot\|_p$ is the L_p-norm, c is a trade-off constant, and $L(\cdot)$ the loss function. The C&W attack is considered among the most powerful attacks, particularly notable for its effectiveness in bypassing many existing defense mechanisms.

Data Processing We iteratively used the three attack presented above, and build a train image set $\mathcal{S}_{train}$ and a test image set $\mathcal{S}_{test}$ both poisoned with adversarial elements selected randomly from respectively train and test dataset.

We define $\mathcal{I}_{train} \subset \mathbb{N}$ and $\mathcal{I}_{test} \subset \mathbb{N}$ such as

$$Card(\mathcal{S}_{train}) = Card(\mathcal{I}_{train})$$

and

$$Card(\mathcal{S}_{test}) = Card(\mathcal{I}_{test})$$

Following the computation of both image set $\mathcal{S}_{train}$ and $\mathcal{S}_{test}$, each element is projected onto the concept bank H by solving the non-negative least squares (NNLS) problem

described in equation 3.11. This results in a matrix $U_i \in \mathbb{R}^{9\times 50}$ for each input sample i, representing its decomposition in the basis defined by H, where each lines $U_{i,:}$ is the projection of a patch of the input i. To make the study more convenient, we take the decomposition of each patchs and retrieve the maximum value for each concepts, and define the vector $u_i \in \mathbb{R}^{50}$ such that $u_i = (max(U_{:,1}), \cdots, max(U_{:,50}))$. We also computed the Sobol indices through the importance estimation method from the CRAFT class, for each elements of $\mathcal{S}_{train}$ and $\mathcal{S}_{test}$, with their predicted class as the class expected. This results in a vector $e_i \in \mathbb{R}^{50}$ for each input i, where $(e_i)_{:,j}$ is the importance value for the j-th concept of H.

Once all projections and all estimations are obtained, the vectors are normalized to ensure consistent scaling across the dataset.

Subsequently, each vectors u_i and e_i are assigned a scalar label $\widehat{y}_i \in \{0, 1\}$, where:

$$\widehat{y}_i = \begin{cases} 0 \text{ indicates that the input is a benign example,} \\ 1 \text{ indicates that the input is an adversarial example.} \end{cases}$$

We conducted then auxiliary sets

$$\mathcal{E}_{train} = \{(e_{train,i}, \widehat{y}_i)\}_{i\in\mathcal{I}_{train}}, \mathcal{U}_{train} = \{(u_{train,i}, \widehat{y}_i)\}_{i\in\mathcal{I}_{train}}$$

and

$$\mathcal{E}_{test} = \{(e_{test,i}, \widehat{y}_i)\}_{i\in\mathcal{I}_{test}}, \mathcal{U}_{test} = \{(u_{test,i}, \widehat{y}_i)\}_{i\in\mathcal{I}_{test}}$$

SVM and RandomForest Hyperparameters In this experimental phase, hyperparameter tuning was performed to enhance the performance of both the Support Vector Machine (SVM) and Random Forest models. For the SVM, the primary hyperparameters considered were the regularization parameter C and the kernel coefficient *gamma*, with the *RBF* kernel selected due to its effectiveness in capturing non-linear relationships. For the Random Forest model, tuning focused on the number of estimators (*n_estimators*) and the maximum depth of the trees (*max_depth*). The hyperparameter optimization was conducted using the scikit-learn framework, employing a two-stage search strategy. First, a RandomizedSearchCV was used to efficiently explore a broad range of hyperparameter values. This was followed by a more focused GridSearchCV, centered on the most promising regions identified in the initial search. Both search procedures were performed using 4-fold cross-validation (cv=4) to ensure robust model evaluation and to reduce the risk of overfitting. In this fine tuning process, both models were trained and tested on the sets defined previously.

1.3 Results & Evaluation

To evaluate models performance, we utilized a confusion matrix. Overall, models trained with projection data demonstrated superior performance compared to those trained using

importance estimation, particularly when adversarial examples were generated using the FGM. Nevertheless, models performance were generally below average when evaluated on adversarial examples detection, regardless of whether they were generated using DeepFool or C&W attacks. It also has to be remark that SVM models outmatched Random Forest Classifier, either when adversarial examples are generated with FGM or DeepFool and C&W methods. (Missing pictures)

2 Areas for Improvement

Several aspects of the current methodology could be improved or further explored. The initial training of the neural network should be conducted with care and precision to ensure optimal performance and the development of meaningful concepts. Secondly, the definition and application of the NNLS problem in the projection phase deserve closer examination. In particular, the distinction between applying the projection to the full-image against processing it in patches, especially when these patches are not normalized using a consistent norm, could lead to meaningful differences in the outcome. These variations might be leveraged to enhance the interpretability or discriminative power of the extracted concepts.

Furthermore, the current approach of using the maximum value across all patches as the final projection output may not optimally preserve relevant information. An alternative strategy could involve normalizing each row of the matrix U by its norm (L_2 seems to be the more convenient), potentially yielding a more balanced representation of the underlying features. For that matter, a more in-depth analysis of the concept space could also be undertaken, particularly to identify redundancies or similarities within the components of H. Such an analysis might reveal opportunities for dimensionality reduction or for grouping semantically similar concepts.

The training process for both the SVM and Random Forest classifiers could also be improved. Indeed, increasing the number of training samples or employing a more precise hyperparameter tuning may enhance classification performance and robustness.

Additionally, a deeper understanding of the nature of the extracted concepts could prove valuable. For instance, analyzing how adversarial exmaples behave in relation to the CRAFT method, and gaining a clearer understanding of Fel's work and related approaches such as the ACE method proposed by Ghorbani et al. [10], or the methodology from Zhang et al. [11], as referenced in Fel's doctoral thesis, could provide useful insights and guide future developments.

Finally, extending the concept discovery process to higher-resolution images, such as those from the ImageNet dataset, could be a promising direction. Although this would significantly increase computational costs, it may allow for the identification of more complexe concepts.

Conclusion

Throughout the course of this internship, I have gained invaluable experience and developed a diverse set of skills that have significantly contributed to both, my academic and professional growth.

One of the key learning outcomes of this internship was the opportunity to explore emerging research domains in machine learning, specifically explainability and the cybersecurity implications of adversarial attacks. These are particularly relevant and increasingly critical areas of study, given the rapid and widespread adoption of AI technologies. While I found these topics intellectually stimulating and highly relevant, I also recognized certain limitations in my background, particularly in mathematics and computer science, which constrained my ability to conduct a more in-depth analysis of adversarial behavior in the context of explainability tools. Nevertheless, the work of Lee et al. [12], which offers a comprehensive review of concept-based methods, provided a useful foundation for understanding this broad and complex field. This literature has helped me appreciate how advancing knowledge in explainable AI could supposedly also support the detection and mitigation of adversarial threats. Moreover, my extensive reading of academic literature during this internship revealed certain gaps in my mathematical foundations. This realization prompted me to revisit and strengthen my understanding of essential topics such as Hilbert spaces and Riemannian geometry. This has been a particularly meaningful aspect of my learning journey, and one for which I am deeply grateful.

Working closely with my colleagues was an enriching experience that greatly enhanced my learning. Interactions with the R&D team deepened my understanding of mathematical concepts, while collaboration with the development and infrastructure teams strengthened my technical competencies. The collaborative environment fostered a dynamic exchange of ideas and offered constant learning opportunities, significantly enriching my grasp of the research process.

Methodologically, observing, through daily meetings, team members such as Sofiane, Paul, Alice, and Perrine was particularly instructive. Their approach to defining and structuring research objectives served as a valuable model for me, enabling me to adopt more effective research practices. Additionally, the guidance of Matthieu, the head of the R&D team, and Sofiane, was instrumental in helping me maintain alignment with project goals and manage the complexity of my tasks. Their feedbacks ensured that I stayed on

track and approached the work with greater strategic clarity.

On a practical level, I made substantial progress in planning and coordinating the various phases of a research project, effectively translating theoretical insights into applied outcomes. I also became more proficient and confident in using key tools and environments such as Python (particularly TensorFlow and PyTorch) and Linux, which are essential in both academic research and industrial applications.

Finally, being embedded within a company provided valuable insight into its organizational dynamics and, more specifically, the pivotal role played by the R&D department in driving innovation and supporting product development. This experience offered me a clearer perspective on how academic research can be translated into industrial applications.

In conclusion, this internship has been an immensely rewarding experience, equipping me with practical tools, technical expertise, and a deeper appreciation of both academic and industrial research environments. It has not only enriched my current knowledge but has also helped refine my professional goals and clarified the direction I wish to pursue in my future career.

Bibliography

[1] T. Fel et al., "Craft: Concept recursive activation factorization for explainability," 2023. arXiv: 2211.10154 [cs.CV]. [Online]. Available: https://arxiv.org/abs/2211.10154

[2] A. C. Serban, E. Poll, and J. Visser, "Adversarial examples - a complete characterisation of the phenomenon," 2019. arXiv: 1810.01185 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1810.01185

[3] C. Szegedy et al., "Intriguing properties of neural networks," 2014. arXiv: 1312.6199 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1312.6199

[4] A. Shafahi, W. R. Huang, C. Studer, S. Feizi, and T. Goldstein, "Are adversarial examples inevitable?," 2020. arXiv: 1809.02104 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1809.02104

[5] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," 2017. arXiv: 1608.04644 [cs.CR]. [Online]. Available: https://arxiv.org/abs/1608.04644

[6] S. Bessai, M. Martel, and A. Ioualalen, "Online detection of adversarial examples by activation profile inspection," 2025.

[7] E. Poeta, G. Ciravegna, E. Pastor, T. Cerquitelli, and E. Baralis, *Concept-based explainable artificial intelligence: A survey*, 2023. arXiv: 2312.12936 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2312.12936

[8] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2015. arXiv: 1412.6572 [stat.ML]. [Online]. Available: https://arxiv.org/abs/1412.6572

[9] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, *Deepfool: A simple and accurate method to fool deep neural networks*, 2016. arXiv: 1511.04599 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1511.04599

[10] A. Ghorbani, J. Wexler, J. Zou, and B. Kim, *Towards automatic concept-based explanations*, 2019. arXiv: 1902.03129 [stat.ML]. [Online]. Available: https://arxiv.org/abs/1902.03129

[11] R. Zhang, P. Madumal, T. Miller, K. A. Ehinger, and B. I. P. Rubinstein, *Invertible concept-based explanations for cnn models with non-negative concept activation vectors*, 2021. arXiv: 2006.15417 [cs.CV]. [Online]. Available: https://arxiv.org/abs/2006.15417

[12] J. H. Lee, G. Mikriukov, G. Schwalbe, S. Wermter, and D. Wolter, *Concept-based explanations in computer vision: Where are we and where could we go?* 2024. arXiv: 2409.13456 [cs.CV]. [Online]. Available: https://arxiv.org/abs/2409.13456

Appendix

NN architecture for CIFAR-100

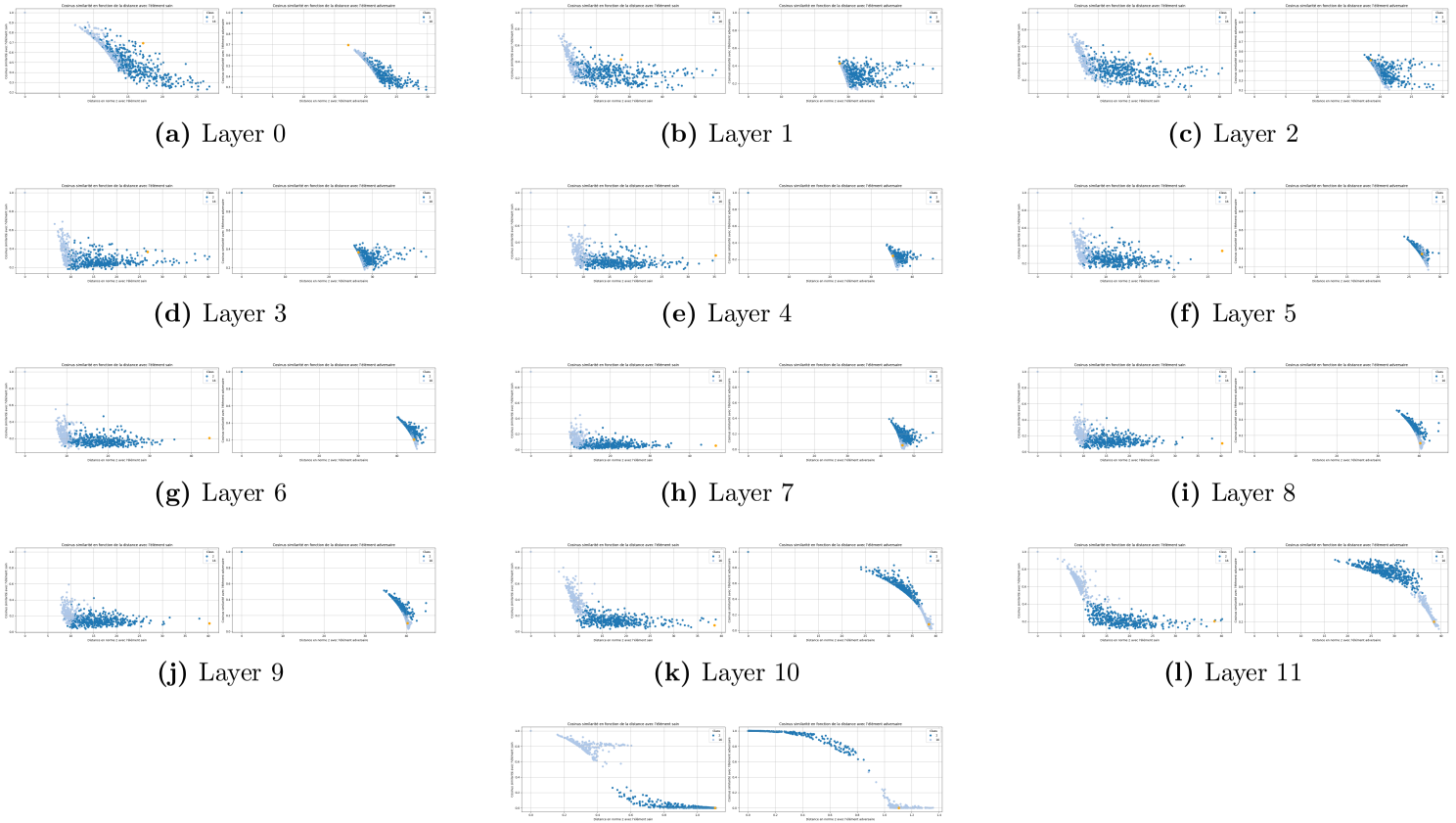
Layer	**Parameters**	**Output Shape**
Input	$32 \times 32 \times 3$	$32 \times 32 \times 3$
Conv2D	64 filters, 3×3, ReLU, same padding	$32 \times 32 \times 64$
Conv2D	64 filters, 3×3, ReLU, same padding	$32 \times 32 \times 64$
MaxPooling	2×2	$16 \times 16 \times 64$
Conv2D	128 filters, 3×3, ReLU, same padding	$16 \times 16 \times 128$
Conv2D	128 filters, 3×3, ReLU, same padding	$16 \times 16 \times 128$
MaxPooling	2×2	$8 \times 8 \times 128$
Conv2D	256 filters, 3×3, ReLU, same padding	$8 \times 8 \times 256$
Conv2D	256 filters, 3×3, ReLU, same padding	$8 \times 8 \times 256$
MaxPooling	2×2	$4 \times 4 \times 256$
Flatten	–	4096
Dense	512 units, ReLU	512
Dense	256 units, ReLU	256
Dense (Softmax)	100 classes	100

Tab. A.1: Convolutional Neural Network architecture for CIFAR-100.

NN architecture for CIFAR-10

Layer	**Parameters**	**Output Shape**
Input	$32 \times 32 \times 3$	$32 \times 32 \times 3$
Conv2D	32 filters, 3×3, ReLU, same padding	$32 \times 32 \times 32$
MaxPooling	2×2	$16 \times 16 \times 32$
Conv2D	64 filters, 3×3, ReLU, same padding	$16 \times 16 \times 64$
MaxPooling	2×2	$8 \times 8 \times 64$
Flatten	–	4096
Dense	128 units, ReLU	128
Dense	64 units, ReLU	64
Dense (Softmax)	10 classes	10

Tab. A.2: Convolutional Neural Network architecture for CIFAR-10.



(a) Layer 0 **(b)** Layer 1 **(c)** Layer 2

(d) Layer 3 **(e)** Layer 4 **(f)** Layer 5

(g) Layer 6 **(h)** Layer 7 **(i)** Layer 8

(j) Layer 9 **(k)** Layer 10 **(l)** Layer 11

(m) Layer 12

Fig. A.1: Cosine similarity by distance across all layers. Each subplot corresponds to one layer, arranged in a 3-column grid for better comparison.

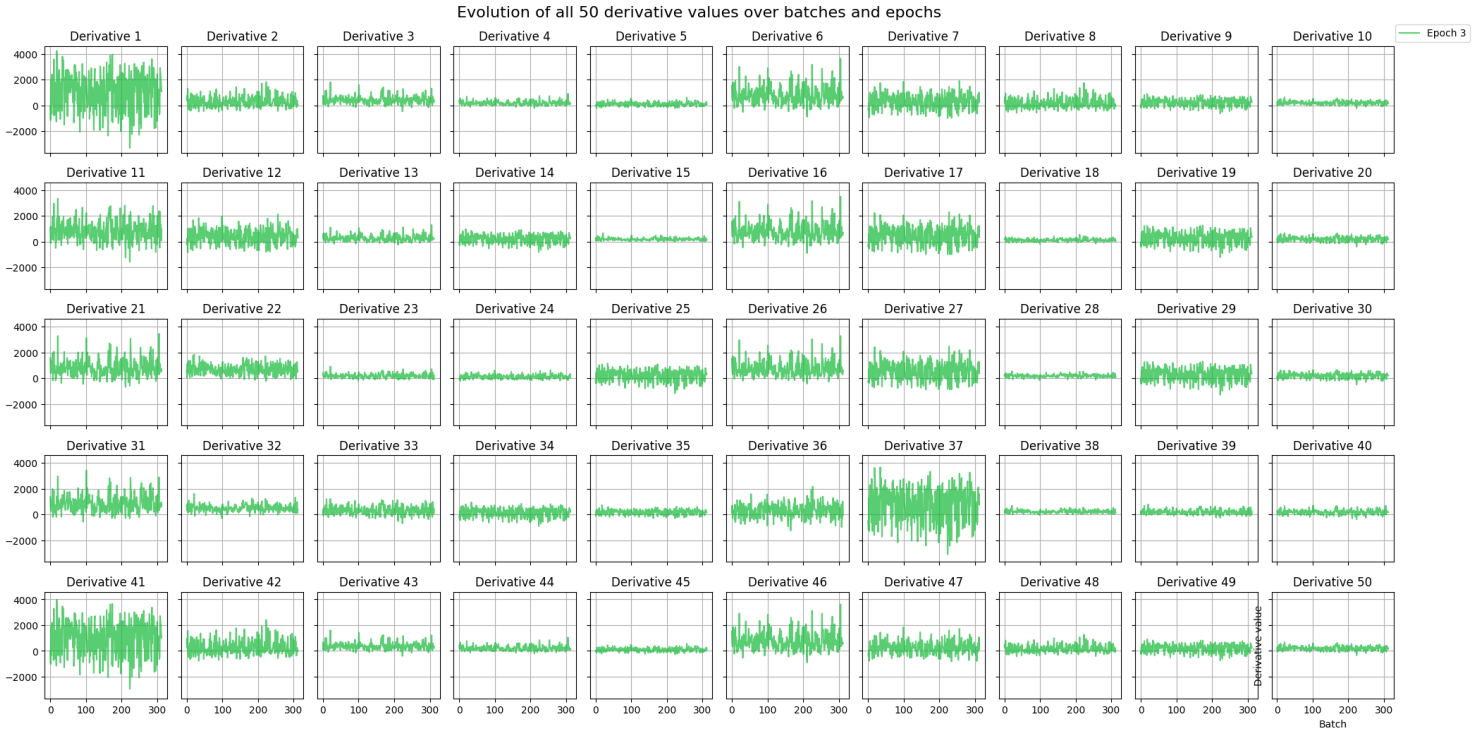


Fig. A.2: **Loss derivative.**

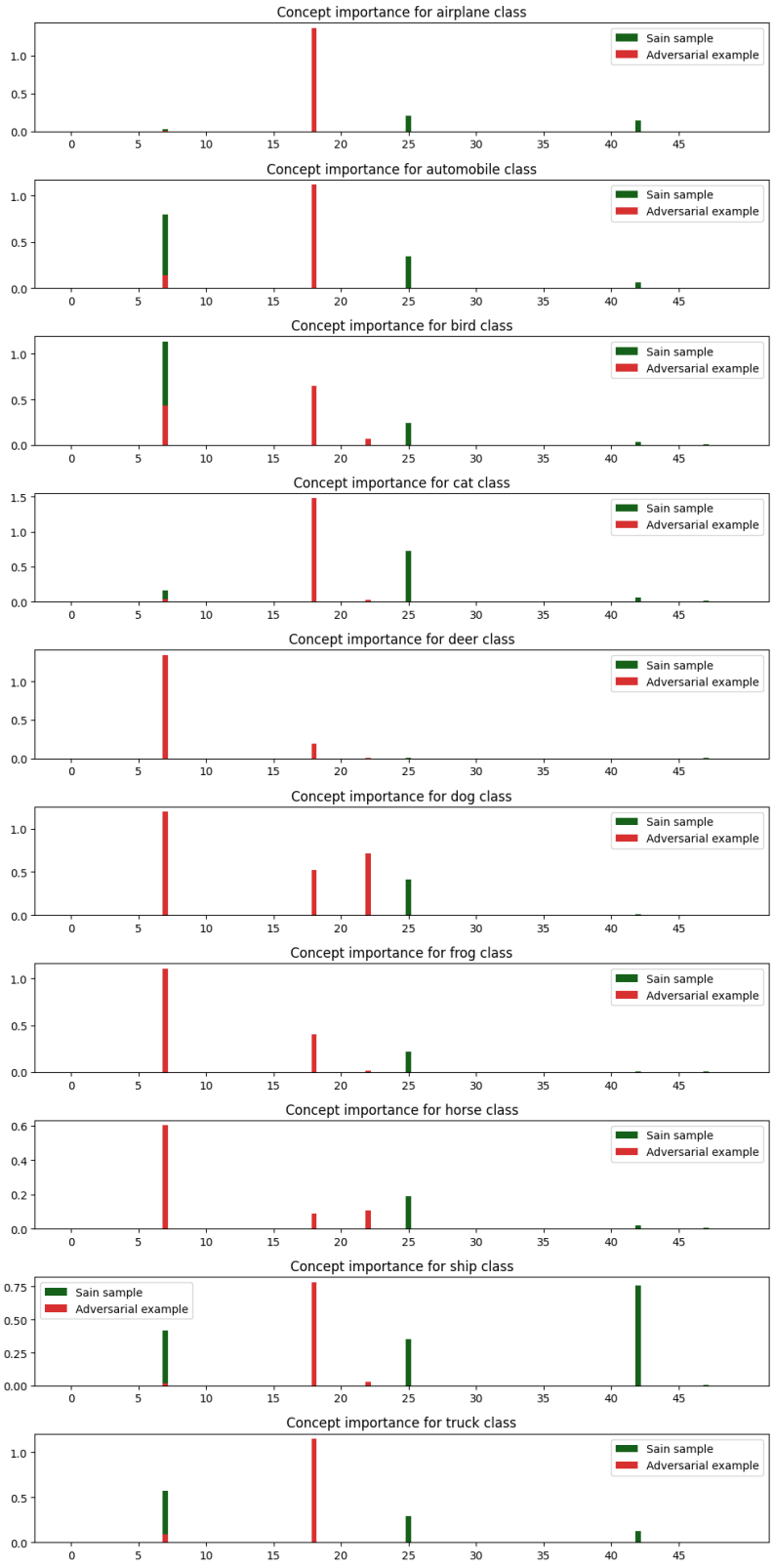


Fig. A.3: **Estimation importance.**

Annexe

Frobenius norm The Frobenius norm is a commonly used matrix norm, defined for any matrix $A \in \mathbb{R}^{m \times n}$ as

$$\|A\|_F = \sqrt{\sum_{i=1}^{m} \sum_{j=1}^{n} |a_{ij}|^2} = \sqrt{\text{trace}(A^\top A)}. \tag{B.1}$$

This norm can be interpreted as the Euclidean norm of the matrix when viewed as a vector in $\mathbb{R}^{mn}$. It is invariant under orthogonal transformations and plays a key role in various matrix approximation problems.

Singular Value Decomposition (SVD) The Singular Value Decomposition is a fundamental matrix factorization technique that expresses any real matrix $A \in \mathbb{R}^{m \times n}$ as a product of three matrices:

$$A = U \Sigma V^\top, \tag{B.2}$$

where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal matrices, and $\Sigma \in \mathbb{R}^{m \times n}$ is a diagonal matrix containing the singular values of A, arranged in non-increasing order. The columns of U and V correspond to the left and right singular vectors of A, respectively. SVD provides valuable insights into the rank, range, and null space of the matrix and is extensively used in numerical linear algebra, signal processing, and data analysis.

Principal Component Analysis (PCA) Principal Component Analysis is a widely employed statistical technique for dimensionality reduction and feature extraction. Given a data matrix $X \in \mathbb{R}^{m \times n}$, where each row corresponds to an observation and each column to a variable, PCA seeks an orthogonal transformation that projects the data onto a lower-dimensional subspace while maximizing the variance of the projected data.

Mathematically, PCA is often computed via the SVD of the centered data matrix $\tilde{X}$ (obtained by subtracting the empirical mean of each variable from the data). The principal components correspond to the right singular vectors of $\tilde{X}$, and the singular values relate to the explained variance along each principal component. This method facilitates noise

reduction, data compression, and visualization of high-dimensional datasets by retaining only the most significant components.

SVM Hyperparameters The following are the key hyperparameters used in the `SVC` implementation of Support Vector Machines in the Scikit-learn library:

- `C`: Regularization parameter. Controls the trade-off between a smooth decision boundary and classifying training points correctly.
- `kernel`: Specifies the kernel type to be used in the algorithm. Options include `'linear'`, `'poly'`, `'rbf'`, `'sigmoid'`, or a callable.
- `degree`: Degree of the polynomial kernel function (`'poly'` kernel only). Ignored by other kernels.
- `gamma`: Kernel coefficient for `'rbf'`, `'poly'`, and `'sigmoid'`. Accepts values: `'scale'`, `'auto'`, or a float.

Random Forest Hyperparameters The following are the main hyperparameters available in Scikit-learn's implementation of the Random Forest classifier:

- `n_estimators`: The number of trees in the forest. Increasing this typically improves performance but also increases training time.
- `max_depth`: The maximum depth of each tree. Limits tree growth to prevent overfitting. If `None`, nodes are expanded until all leaves are pure.
- `min_samples_split`: The minimum number of samples required to split an internal node. Can be an int or a float (fraction of total samples).
- `min_samples_leaf`: The minimum number of samples required to be at a leaf node. Helps control overfitting.
- `max_features`: The number of features to consider when looking for the best split. Supports `'sqrt'`, `'log2'`, float (fraction), int, or `None`.
- `max_leaf_nodes`: Grow trees with at most this many leaf nodes. If `None`, then unlimited.
- `max_samples`: If bootstrap is `True`, the number or fraction of samples to draw from X to train each base estimator.